

COMP 330 PROFESSIONAL PAPER SERIES

Why Software Engineering Matters More in the AI Era

A Professional Orientation for COMP 330

William T. O'Connell, Ph.D.

Adjunct Professor of Computer Science, Loyola University Chicago

Former IBM Distinguished Engineer and CTO, IBM Consulting Quality and Delivery

Creator, Engineering Trustworthy Intelligent Systems (ETIS)

COMP-WP-001 | Version 1.0 | Fall 2026

Core Thesis

Artificial intelligence is making software easier to produce and harder to trust. The engineers who matter most will not be those who can generate the most code. They will be the people who can turn accelerated production into systems that are understandable, reviewable, testable, secure, operable, and accountable.

Read this before Class 1. You are not expected to memorize every term. The goal is to understand the direction of the profession, the standard this course will hold, and why the work ahead should be both demanding and exciting.

Executive Abstract

Software engineering is entering a period of rapid change. Generative AI can already explain unfamiliar code, produce implementations, draft tests, propose architectures, review pull requests, and create documentation. Coding agents can accept an issue, inspect a repository, modify multiple files, run tools, execute tests, and return a pull request for human review. The near-term direction is clear: engineers will increasingly work with AI systems that do more than suggest the next line of code. They will delegate bounded engineering work to systems that can plan and act.

This does not make software engineering obsolete. It changes where the difficult work lives. When code becomes cheaper to produce, the scarce capabilities become understanding the real problem, expressing precise intent, designing coherent systems, supplying authoritative context, evaluating generated work, controlling authority, managing risk, operating production systems, and defending decisions with evidence. DORA's 2025 research describes AI as an amplifier: it magnifies the strengths of effective engineering organizations and the weaknesses of struggling ones [3]. Faster generation can therefore produce faster learning—or faster confusion.

The future belongs neither to engineers who reject AI nor to people who accept its output passively. It belongs to engineers who can use AI aggressively while remaining responsible for the system. That is the professional standard of COMP 330. You will work in teams, use a GitHub repository as the engineering system of record, use AI throughout the lifecycle, and build one project through two development cycles. Your demonstrations matter, but your engineering evidence matters more: requirements, decisions, issues, pull requests, reviews, tests, release records, operational thinking, and honest disclosure of limitations.

This paper introduces the central idea of the course: working code is not the same as engineered software. A demo shows that one path worked once. Professional engineering asks whether the system can be understood, changed, reviewed, tested, governed, recovered, and trusted under real conditions. AI raises that standard because it increases both what teams can create and how quickly weak assumptions can become operational consequences.

The Question for Your Generation

When intelligent tools can generate artifacts faster than any individual can inspect them, what distinguishes a professional software engineer? The answer is not typing speed. It is judgment, system ownership, disciplined verification, and the ability to produce trustworthy change.

1. You Are Entering the Profession at an Inflection Point

Every generation of software engineers enters a profession shaped by a major transition. Earlier generations adapted to personal computing, graphical interfaces, the Internet, open source, mobile platforms, cloud computing, Agile delivery, DevOps, and continuous deployment. Your generation enters as artificial intelligence becomes both part of the software being built and part of the workforce building it.

The visible change is code generation. The deeper change is delegated engineering. GitHub's coding agent can work from an assigned issue, execute in a development environment, and open a pull request. OpenAI describes Codex as a software-engineering agent that can work on tasks in parallel, modify repositories, run tests, fix bugs, and propose changes for review [4][5]. GitHub has also moved toward repository instructions, specialized agents, and a unified environment for orchestrating multiple agents [6][7]. These products will keep changing, but the direction is more important than any particular tool.

The unit of work is moving from a prompt-response interaction toward a delegated assignment. A human states the desired outcome, supplies constraints and context, and authorizes an agent to perform a sequence of actions. The

agent may choose files, commands, dependencies, tests, or implementation approaches that were not individually specified. The engineer’s job therefore expands from producing the artifact to governing the path by which the artifact is produced.

This transition is exciting because it increases your reach. A small team can investigate more alternatives, automate more routine work, test more scenarios, and work across a larger system than before. It is demanding because the cost of weak intent and weak review also falls. An ambiguous requirement can be implemented in minutes. A poor design can be propagated across many files. A plausible but incorrect test can create false confidence. A large change can appear polished before anyone understands it.

The professional opportunity is enormous. The World Economic Forum’s 2025 analysis places AI and big data, cybersecurity, and technological literacy among the fastest-growing skill areas, while also emphasizing creative thinking, resilience, curiosity, leadership, and collaboration [8][9]. The engineers who succeed will combine technical capability with the human skills needed to frame problems, challenge assumptions, communicate decisions, and own consequences.

A Better Career Question

Do not ask, “Will AI replace software engineers?” Ask, “Which engineering responsibilities become more valuable when software production accelerates?”

2. Coding Is Essential—but Coding Is Not the Whole Profession

Programming is the ability to express a solution in executable form. Software engineering is the discipline of creating and sustaining systems under real constraints. The distinction is not an insult to programming. Strong engineers must understand code deeply. The distinction recognizes that production software must survive far more than a successful execution.

A real system has stakeholders, changing requirements, architecture, interfaces, dependencies, data, security boundaries, users, operational environments, cost, failure modes, and people who must maintain it after the original developers move on. A local implementation can be correct while the system is wrong. The feature may solve the wrong problem, violate an architectural boundary, expose protected data, fail under concurrency, create an unrecoverable migration, or be impossible for another team to operate.

AI makes the distinction sharper. If the first draft of code is inexpensive, the value shifts toward deciding what should be built and determining whether the generated result belongs in the system. Foundations still matter because you cannot review what you do not understand. Data structures, algorithms, programming languages, databases, networking, operating systems, concurrency, security, and architecture give you the mental models required to detect when a plausible answer is technically or operationally wrong.

The engineer of the AI era must work at two levels simultaneously. At the component level, you must understand logic, state, interfaces, failure, and performance. At the system level, you must understand purpose, boundaries, evidence, risk, deployment, users, and change over time. AI can help at both levels, but it cannot accept professional responsibility for either.

Programming question	Engineering question
Does the code run?	Does the system satisfy the real requirement under relevant conditions?
Is the function correct?	Does the change fit the architecture, security model, and operational environment?
Can the feature be demonstrated?	Can the team support its release claim with traceable evidence?
Can AI generate the artifact?	Can the team verify, explain, maintain, and own the result?

Can the system be deployed?

Can it be observed, recovered, governed, and sustained in production?

3. AI Raises the Engineering Bar

The easiest interpretation of AI is that it lowers the bar: less code must be written manually, fewer details must be remembered, and more work can be completed quickly. That interpretation confuses production with engineering. AI lowers the cost of producing candidate artifacts. It raises the burden of selection, verification, integration, and accountability.

DORA's State of AI-assisted Software Development report describes AI as an amplifier of the surrounding organizational system [3]. Strong teams can use it to shorten feedback and expand capability. Weak teams may create more instability because unclear priorities, poor data, fragile architecture, low-quality reviews, and weak delivery practices are accelerated rather than repaired. The lesson applies directly to your work: AI does not rescue a weak engineering process. It makes that weakness move faster.

Verification becomes a larger part of the job. Generated code may compile and still misunderstand the requirement. Generated tests may simply confirm the generated implementation. Generated documentation may describe what should exist rather than what actually exists. Generated architecture may use fashionable patterns without fitting the constraints. The more polished the output, the easier it is to confuse fluency with correctness.

Context also becomes part of engineering. Coding agents rely on repository structure, requirements, architecture decisions, naming conventions, test commands, policies, and examples. A poorly maintained repository creates poor context for both humans and AI. Documentation, issue quality, test quality, and architectural clarity become productivity infrastructure rather than paperwork.

Finally, authority becomes an engineering concern. An AI assistant that drafts text is different from an agent that runs commands, changes files, modifies infrastructure, or acts in production. As tools gain authority, teams must define identity, permissions, review points, evidence, and intervention. The defining question is not only what the agent can do. It is what the organization has responsibly authorized it to do.

Core Course Principle

AI may accelerate the work, but the team remains accountable for every requirement, decision, change, test, limitation, release claim, and operational consequence.

4. The Model Is Not the System

Public discussion often treats an AI model as though it were the intelligent system. In practice, the model is one component. The deployed system includes requirements, application code, data, prompts, retrieval, context, memory, tools, identities, permissions, interfaces, workflows, tests, policies, human review, telemetry, and operational procedures. Trust depends on how these elements interact.

A capable model can produce unsafe behavior when the surrounding system supplies stale information, grants excessive permissions, hides uncertainty, or lacks meaningful intervention. A less capable model can be useful in a tightly bounded workflow with strong verification. This is why NIST defines trustworthy AI through multiple characteristics—validity and reliability, safety, security and resilience, accountability and transparency, explainability, privacy, and fairness—rather than through one performance score [10][11].

The same systems principle applies to conventional software. A correct algorithm can be embedded in an unreliable service. Secure code can be deployed through an insecure pipeline. A well-tested feature can fail because of an undocumented dependency. Professional engineering is the discipline of connecting the parts strongly enough that the complete system supports the claims being made about it.

In COMP 330, you will repeatedly be asked to move beyond the artifact. When you present a requirement, you must connect it to the project intent. When you present architecture, you must explain boundaries and tradeoffs. When you show code, you must connect it to an issue, review, and tests. When you demonstrate a release, you must identify what remains weak and what evidence supports readiness.

5. The Repository Becomes Your Engineering Workspace

Most university courses use a learning management system as the center of the class. Sakai remains the official place for announcements, schedules, assignments, grades, and course guidance. It is not the place where professional software engineering happens. Your GitHub repository will be the operational engineering environment for your team.

A repository is not merely a code container. It is where intent becomes controlled change. Requirements and project decisions establish why the work exists. Issues turn intent into bounded assignments. Branches isolate change. Pull requests expose the engineering argument. Reviews challenge the work. Automated checks provide evidence. Releases establish baselines. Runbooks, defects, postmortems, and operational observations preserve what the team learns.

This matters for your grade, but more importantly it matters for your professional development. A future interviewer cannot observe every hour you spent working. A strong repository can show how you think: how you decompose work, communicate scope, make decisions, respond to review, test claims, disclose AI use, and improve a system over time. The repository becomes part of your professional story.

Repository-centered work also creates the context that AI tools need. Clear requirements, current architecture, useful README guidance, meaningful issues, and executable tests help agents produce better work. The course will therefore treat repository quality as both human communication and machine-readable engineering context.

Repository Doctrine

Sakai explains the course. GitHub preserves the engineering record. ETIS supplies the broader professional discipline. AI accelerates the work—but the team owns the result.

6. Evidence Is How Engineers Make Trustworthy Claims

Every important engineering statement is a claim. “The requirement is understood.” “The design is appropriate.” “The change is safe enough to merge.” “The release is ready.” “The system can recover.” A professional claim is not accepted because the team sounds confident. It is accepted because relevant evidence supports it.

Evidence is not the same as artifact volume. A long document may provide little proof. A small number of well-connected artifacts may be persuasive. A requirement linked to an issue, a pull request, focused tests, review findings, and a release record creates a stronger chain than five disconnected documents. The quality of the relationship matters more than the number of files.

This is especially important when AI can generate documentation, test plans, diagrams, and code. The presence of an artifact no longer proves that the team understood or used it. You must be able to show what decision it supported, how it changed the work, what source was authoritative, what verification occurred, and what uncertainty remains.

Your presentations will therefore be evidence-based defenses. You will show the system, but you will also navigate the repository. You should be prepared to explain what changed, why it changed, how it was reviewed, how it was tested, what AI contributed, what risks remain, and why the team believes the current release is appropriate for its stated boundary.

Engineering claim	Useful evidence
We understood the requirement.	Questions, assumptions, acceptance criteria, scope decisions, traceable issue.
The architecture is suitable.	Architecture views, ADRs, alternatives, boundaries, risks, reviewer feedback.
The change is ready to merge.	Bounded pull request, linked issue, tests, CI results, review disposition.
AI was used responsibly.	AI-use disclosure, accepted/rejected output, human changes, independent verification.
The release is ready.	Known baseline, test evidence, limitations, release notes, rollback and operational guidance.
The team learned from experience.	Defect history, postmortem, changed tests, revised decisions, Cycle 2 improvements.

7. Professional Teamwork Is Not Optional

Software engineering is a team sport. Modern systems are too broad, interconnected, and consequential to depend on one person's private understanding. Teams coordinate product intent, architecture, implementation, security, quality, operations, and stakeholder communication. Even when one engineer writes most of a component, others must be able to review, operate, and change it.

In this course, every team member is a developer, and team members will also hold assigned engineering roles. Roles such as team lead, project manager, architecture owner, quality or test lead, repository or release owner, and AI-governance owner are not honorary titles. They make accountability visible. The role owner ensures that the concern is addressed, coordinated, and evidenced; the entire team still shares responsibility for the system.

Professional teamwork includes disagreement. A good review is not a compliment ceremony. It is structured challenge intended to improve the work. A reviewer may question a requirement, identify hidden complexity, reject an unsafe change, request better tests, or expose a risk the author missed. The standard is not perfect agreement. It is respectful, evidence-based reasoning with a clear disposition.

AI can improve team coordination by drafting issues, summarizing discussion, generating review candidates, or identifying dependencies. It can also overwhelm the team with output that no one has time to understand. The answer is small batches, clear ownership, meaningful pull requests, and review capacity that grows with production capacity.

Professional Habit

A team is not strong because its members divide the work and disappear. A strong team maintains shared understanding, reviews one another's work honestly, and can explain the system without relying on one person.

8. Your Project Will Mature Through Two Engineering Cycles

COMP 330 is organized around one team project that matures through the semester. Requirements, architecture, issues, code, tests, reviews, releases, operational evidence, and postmortems will accumulate around the same system rather than appear as unrelated submissions.

Cycle 1 asks whether the team can move from intent to a controlled, reviewable release. You will establish the repository, clarify requirements, design and implement a bounded vertical slice, verify its behavior, and defend the evidence supporting release readiness. The objective is controlled engineering, not maximum feature count.

Cycle 2 asks whether the team can improve the system based on evidence. You will examine defects, weak tests, review findings, operational gaps, and limitations from Cycle 1, then target the improvements that matter most—possibly a feature, but often better recovery, observability, security, traceability, maintainability, or release discipline.

This reflects professional reality: engineering does not end when the demo works. The ability to learn from evidence and improve a system is a stronger professional signal than producing a one-time prototype.

Cycle	Primary question	What success looks like
Cycle 1	Can the team build and release a bounded capability under engineering control?	Clear intent, coherent architecture, traceable change, credible tests, reviewed evidence, honest limitations.
Cycle 2	Can the team improve what the evidence says is weak?	Learning from defects and review, targeted maturity improvements, stronger operational and governance evidence.

9. How to Use AI Professionally in This Course

AI tools are encouraged in COMP 330. Avoiding them would create an artificial environment that does not reflect the profession you are entering. The expectation is not avoidance. The expectation is responsible use, verification, disclosure, and human ownership.

Use AI to explore alternatives, explain unfamiliar code, draft requirements, generate test ideas, review changes, find edge cases, improve documentation, or help you navigate tools. But do not confuse assistance with authority. You must decide what to accept, what to reject, what to modify, and how to verify it. The more consequential the output, the stronger the required review.

Disclosure is treated as provenance rather than confession. The useful record is not merely “AI was used.” It is which tool was used, for what task, what context it received, what output was accepted or rejected, what the team changed, how the result was verified, and who owns the decision. This allows the team and instructor to evaluate judgment rather than police tool use.

There is also a learning obligation. If AI produces an implementation you cannot explain, you do not yet own the work. If it generates a test you cannot justify, the test is not credible evidence. If it recommends architecture you cannot defend, the recommendation has not become an engineering decision. AI should expand your capability, not hide the absence of understanding.

The strongest AI users will not be the students who prompt most often. They will be the students who frame better questions, supply better context, compare alternatives, detect weak output, and integrate assistance into a disciplined engineering workflow.

AI Use Standard

AI proposes; engineers verify. AI accelerates; engineers remain accountable. AI can produce the first draft; the team must produce the engineering judgment.

10. What Employers Will Need from You

Routine implementation work will continue to change. Some tasks that once occupied junior engineers will be automated or compressed. That does not eliminate entry-level opportunity, but it changes the evidence employers will seek. A candidate who can only produce code that resembles a tutorial is easier to substitute. A candidate who can understand a system, communicate intent, use AI effectively, review change, reason about failure, and work professionally on a team is harder to replace.

The World Economic Forum identifies AI and big data, cybersecurity, and technological literacy as rapidly growing skills, alongside creative thinking, resilience, flexibility, curiosity, and lifelong learning [8][9]. CS2023 similarly moves beyond topic coverage toward competencies and professional dispositions, emphasizing collaboration, professional responsibility, ethics, and the ability to apply and evaluate knowledge [12][13]. ABET's current software-engineering criteria continue to require requirements analysis, design and construction, security, verification, validation, engineering processes, and tools for complex systems [14].

These directions reinforce the same message: the profession needs people who can combine technical knowledge with judgment. You will need to learn new tools repeatedly. Specific AI products will change. The durable capability is knowing how to learn, how to assess a tool, how to integrate it safely, and how to remain responsible when the tool produces work at a scale you could not create manually.

Your strongest portfolio evidence will therefore be more than a finished application. It will show the path from intent to outcome: meaningful issues, architecture decisions, reviewable pull requests, tests tied to claims, professional communication, release evidence, and improvement across cycles. That record demonstrates how you work when the answer is not already known.

11. What Success in COMP 330 Looks Like

This course is not designed to reward the team that produces the most artifacts or the most features. It rewards teams that can make defensible engineering claims. A small, coherent system with visible decisions, strong tests, honest limitations, and professional review is better engineering than a larger system no one fully understands.

Success begins with preparation and participation. Review the current Sakai module, read the assigned material, understand the weekly objective, and arrive ready to make decisions with your team. Keep the repository current. Build evidence continuously rather than reconstructing it before a deadline. Ask questions while ambiguity can still be corrected.

Success also requires honesty. Surface defects, risks, uncertainty, missed estimates, AI limitations, and weak evidence. Concealing a problem does not improve the system; it merely delays learning. Professional teams are trusted because they make risk visible early enough to act.

Finally, success requires ownership. You should be able to navigate your team's repository, explain the decisions relevant to your role, show supporting evidence, and describe what you would improve next. You do not need to know everything individually. You do need to understand enough of the complete system to participate responsibly in review and release.

Professional behavior	What it looks like in the course
Build evidence continuously.	Issues, decisions, reviews, tests, and release records evolve with the work.
Use AI responsibly.	AI use is purposeful, disclosed, verified, and owned by the team.
Review honestly.	Reviews challenge assumptions and lead to changes or explicit decisions.
Operate with accountability.	Roles are active, risks have owners, and limitations remain visible.

Communicate professionally.

The repository and team discussions are clear enough for another engineer to follow.

Improve the system.

Cycle 2 responds to evidence rather than merely adding features.

12. A Career Lesson—and an Invitation

Throughout my career, I learned that engineering teams rarely fail because no one can write code. They fail because intent remains ambiguous, decisions are not communicated, architecture erodes, assumptions go untested, risks remain hidden, review becomes ceremonial, and operations are treated as someone else’s problem. The technology changes. Those failure patterns endure.

AI will not remove those responsibilities. It will expose them more quickly. Teams with strong discipline will use AI to achieve things that were previously impractical. Teams without that discipline may create more software and less understanding. The difference will be the quality of the engineering system around the tools.

That is why this course is different from a programming course. You will still write code. You will also learn how professional teams create shared intent, control change, preserve evidence, use AI without surrendering judgment, and mature a system through review and operational learning.

The work will be demanding because the standard is real. It should also be energizing. You are entering the profession at a moment when the definition of an engineer is expanding. You have the opportunity to become part of the first generation trained not merely to code with AI, but to engineer trustworthy systems with it.

The COMP 330 Commitment

The goal is not merely to build software. The goal is to build software that can survive review, failure, operational reality, and change—and to become the kind of engineer who can explain why it should be trusted.

Before Class 1

After reading this paper, complete the following orientation steps. You are not expected to understand every resource in depth yet. You should know where the course’s operating guidance and engineering resources live.

- ✓ Watch the COMP 330 course-orientation video.
- ✓ Read the Start Here page and understand the different roles of Sakai, GitHub, ETIS, and AI tools.
- ✓ Open the Engineering Resources page and locate the COMP 330 Repository Starter Kit, setup guidance, AI-use expectations, and review materials.
- ✓ Review the Why This Matters page and come prepared to discuss why working code is not the same as engineered software.
- ✓ Be ready to answer this question: If AI makes software easier to produce, why does software engineering matter more?

Conclusion

Artificial intelligence is not ending software engineering. It is ending the period in which artifact production could stand in for engineering competence. Code, tests, documents, and diagrams can now be generated rapidly. What remains scarce is the ability to define the right problem, design a coherent system, supply trustworthy context, verify behavior, govern authority, operate responsibly, and defend decisions with evidence.

That is the opportunity in front of you. Learn to use AI as leverage, not as a substitute for understanding. Learn to treat the repository as an engineering workspace, not a file dump. Learn to review what appears polished, expose what remains uncertain, and improve what operational evidence reveals.

The bar is rising. That is good news for students who are willing to become engineers rather than artifact producers. COMP 330 is designed to help you cross that bar.

References and Further Reading

- [1] William T. O’Connell. Engineering Trustworthy Software in the AI Era. ETIS White Paper WP-001, Revised Publication Edition, 2026. <https://etisframework.org>
- [2] William T. O’Connell. Engineering Education in the AI Era. ETIS White Paper WP-005, 2026. <https://etisframework.org>
- [3] DORA. State of AI-assisted Software Development 2025. <https://dora.dev/dora-report-2025/>
- [4] GitHub. GitHub Copilot: Meet the New Coding Agent, 2025. <https://github.blog/news-insights/product-news/github-copilot-meet-the-new-coding-agent/>
- [5] OpenAI. Introducing Codex, 2025. <https://openai.com/index/introducing-codex/>
- [6] GitHub. Copilot Coding Agent Is Now Generally Available, 2025. <https://github.blog/changelog/2025-09-25-copilot-coding-agent-is-now-generally-available/>
- [7] GitHub. Introducing Agent HQ: Any Agent, Any Way You Work, 2025. <https://github.blog/news-insights/company-news/welcome-home-agents/>
- [8] World Economic Forum. The Future of Jobs Report 2025. <https://www.weforum.org/publications/the-future-of-jobs-report-2025/>
- [9] World Economic Forum. Future of Jobs Report 2025: The Jobs of the Future and the Skills You Need, 2025. <https://www.weforum.org/stories/2025/01/future-of-jobs-report-2025-jobs-of-the-future-and-the-skills-you-need-to-get-them/>
- [10] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0), 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- [11] National Institute of Standards and Technology. Trustworthy and Responsible AI. <https://www.nist.gov/trustworthy-and-responsible-ai>
- [12] ACM, IEEE Computer Society, and AAAI. Computer Science Curricula 2023. <https://csed.acm.org/final-report/>
- [13] ACM, IEEE Computer Society, and AAAI. CS2023 Body of Knowledge and Competency Framework. <https://csed.acm.org/wp-content/uploads/2024/01/Body-of-Knowledge-v1-bookmarksv2.pdf>
- [14] ABET. Criteria for Accrediting Engineering Programs, 2025–2026: Software and Similarly Named Engineering Programs. <https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-engineering-programs-2025-2026/>
- [15] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1, 2024. <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>

About the Author

William T. O’Connell, Ph.D., is an adjunct professor of computer science at Loyola University Chicago and the creator of Engineering Trustworthy Intelligent Systems (ETIS). He brings four decades of experience in software engineering, enterprise architecture, product and platform development, consulting delivery, quality, and technology leadership.

His career includes work across AT&T Bell Laboratories, IBM Research, IBM product development, enterprise software, major client modernization programs, and global technical leadership. He served as an IBM Distinguished Engineer and as CTO for IBM Consulting Quality and Delivery. His teaching focuses on helping students combine strong technical foundations with professional judgment, responsible AI use, evidence, teamwork, and operational accountability.